任务一　基于神经渲染的街景三维重建技术

子任务三

[redacted]

(申请优秀营员)

2024年7月18日

目录

1 子任务三：数据下载与训练复现 **2**
1.1 数据预处理 . . . 2
1.2 超参选择 . . . 3
1.3 实验 . . . 4
1.4 感知实例编辑 . . . 7
1.5 潜在改进方向与建议 . . . 7
1.6 总结 . . . 8

附录 **10**

A 附录1：环境安装 **10**

B 附录2：超参数详细配置 **10**

C 附录3：MARS Pipeline 详细解释 **12**

D 附录4：消融实验模型配置 **19**
D.1 VKITTI_Recon_Mars_Car_Depth_Semantic(ID6) . . . 19
D.1.1 方法名称和训练配置 . . . 19
D.1.2 数据管理器配置 . . . 19
D.1.3 模型配置 . . . 20
D.1.4 优化器配置 . . . 20
D.2 VKITTI_Recon_Mars_Car_Depth(ID4) . . . 22
D.2.1 方法名称和训练配置 . . . 22
D.2.2 数据管理器配置 . . . 22
D.2.3 模型配置 . . . 22
D.2.4 优化器配置 . . . 22
D.3 KITTI_Recon_Mars_Car_Depth_Semantic(ID6) . . . 24
D.3.1 方法名称和训练配置 . . . 24
D.3.2 数据管理器配置 . . . 24
D.3.3 模型配置 . . . 24
D.3.4 优化器配置 . . . 24
D.4 KITTI_Recon_Mars_Car_Depth(ID4) . . . 26
D.4.1 方法名称和训练配置 . . . 26
D.4.2 数据管理器配置 . . . 26
D.4.3 模型配置 . . . 26
D.4.4 优化器配置 . . . 26

1 子任务三：数据下载与训练复现

1.1 数据预处理

单目深度图的生成

本文使用 mini-omnidata[4] 来生成 KITTI 数据集的深度图。通过修改 mini-omnidata/run.py 中的核心代码以优化深度图生成的效果，目的是将 RGB 图像进行颜色映射和归一化处理，并生成合适的图像张量：

Listing 1: colorize Function

```
def colorize(image, cmap="turbo"):
  h, w, c = image.shape
  print(h, w, c)
  if c == 1:  # depth
  image = image.squeeze()
  image_normalized=(image-np.min(image))/(np.max(image)-np.min(image))
  cmap = plt.get_cmap(cmap)
  image_colorized = cmap(image_normalized)[:, :, :3]
  return np.uint8(image_colorized * 255)
else:
  return np.uint8(image * 255)
```

Listing 2: raw_image_to_tensor Function

```
  if h_net != h_raw or w_net != w_raw:
    resizer = Resize(
      h_net,
      w_net,
      resize_target=None,
      keep_aspect_ratio=True,
      ensure_multiple_of=self.patch_size,
      resize_method="minimal",
      image_interpolation_method=cv2.INTER_CUBIC,
    )
    im = resizer({"image": im_raw})["image"]
  else:
    im = im_raw
```

调整后的代码生成效果对比如下所示：

如图 1 所示，第一张图为原图，第二张图为使用 mini-omnidata 初步生成的深度图，第三张图为经过优化代码后生成的深度图。因此，深度图的数据预处理得以完成。

整个模型使用 DPTDepthModel 进行深度图生成。DPTDepthModel 结合了 Vision Transformer (ViT) 和 ResNet50 两种架构的优点。具体而言，模型结构如下：

- **Vision Transformer (ViT)**：将输入图像分割成若干小块（patch），并对这些小块进行嵌入和编码处理。ViT 部分包含 12 层 Transformer 编码器，每层由多头自注意力机制和前馈神经网络组成。



图 1: 深度图对比

- **ResNet50**：接收 ViT 提取的全局特征，并通过 50 层卷积层和残差连接提取局部特征。
- **上采样和卷积层**：将特征图恢复到与输入图像相同的分辨率，生成最终的深度图。

输入图像为单张 RGB 图像，大小为 $H \times W \times 3$，经过预处理后调整为 32 的倍数并进行归一化处理。输出为单通道深度图，大小为 $H \times W \times 1$，深度值为浮点数，表示每个像素到摄像头的距离。

推理流程包括图像预处理、特征提取和深度预测、以及结果处理和保存。首先，读取并归一化输入图像，调整图像尺寸以适应模型的输入要求。然后，通过 ViT 提取全局特征，再通过 ResNet50 提取局部特征，最后通过上采样和卷积层生成深度图。预测的深度图被转换为可视化的彩色图像并保存，同时以 .npy 文件格式保存用于后续分析。通过结合 ViT 和 ResNet50，该神经网络模型能够从单张 RGB 图像中有效地提取深度信息，生成精确的深度图。

1.2 超参选择

超参数设置

- **学习率（Learning Rate）**：背景节点为 1×10^{-3}，衰减至 1×10^{-5}；前景节点为 5×10^{-3}，衰减至 1×10^{-5}。
- `near_plane`：光线采样的起始位置，单位是米，设为 0.05。
- `far_plane`：光线采样的结束位置，单位是米，设为 1000.0。
- `latent_size`：每个类别对象的潜在向量的大小，设为 256。
- `depth_loss_mult`：深度损失的乘数，设为 1×10^{-2}。
- `semantic_loss_mult`：语义损失的乘数，设为 1.0。
- `depth_sigma`：深度值的不确定性，单位是米，设为 0.05。
- `starting_depth_sigma`：初始深度值的不确定性，设为 4.0。
- `sigma_decay_rate`：指数衰减速率，设为 0.99980。

- `depth_loss_type`：深度损失的类型，设为 `DS_NeRF`。
- `background_color`：背景颜色配置，可以是 `"random"`、`"last_sample"` 或 `"black"`，设为 `"black"`。

其他超参数配置见附录 B 。

损失函数参数

1. **颜色损失** ($\mathcal{L}_{\text{color}}$)：默认MSE为1.0
2. **深度损失** ($\mathcal{L}_{\text{depth}}$)：在不同的配置中，深度损失的乘数如下：

- `mono_depth_loss_mult:0.01`
- `depth_loss_mult:0.0` 或 `depth_loss_mult:1e-2`（0.01）

3. **语义损失** ($\mathcal{L}_{\text{sem}}$)：语义损失的乘数如下：

- `semantic_loss_mult: 1.0` `semantic_loss_weight: 0.1`（在特定背景模型配置中）

4. **天空正则化损失** ($\mathcal{L}_{\text{sky}}$)：

loss_dict["sky_mask_loss"] = binary_cross_entropy(occ, 1 − sky_mask.float()) + occ[sky_mask].mean()

loss_dict["sky_color_loss"] = self.rgb_loss(outputs["sky_rgb"][sky_mask], image[sky_mask])

均为1.0

5. **累积密度损失** ($\mathcal{L}_{\text{accum}}$)：

`interlevel_loss_mult: float = 1.0`

1.3 实验

在实验开始之前，我给出了Mars Pipeline 的详细解释，见附录 C 。

根据配置的模型参数，背景节点和前景节点分别采用不同的NeRF框架。具体来说，背景节点使用基于网格的 NeRF 和 proposal sampler ；前景节点使用改进的类别级表征和 c2f 的采样策略。针对KITTI数据集，我加载了预训练模型，并在其基础上进行了30万次迭代。渲染结果如图2所示。评价指标包括 PSNR 、SSIM 和 LPIPS，实验结果和渲染组合图如下：

PSNR	SSIM	LPIPS
27.945	0.8999	0.05458

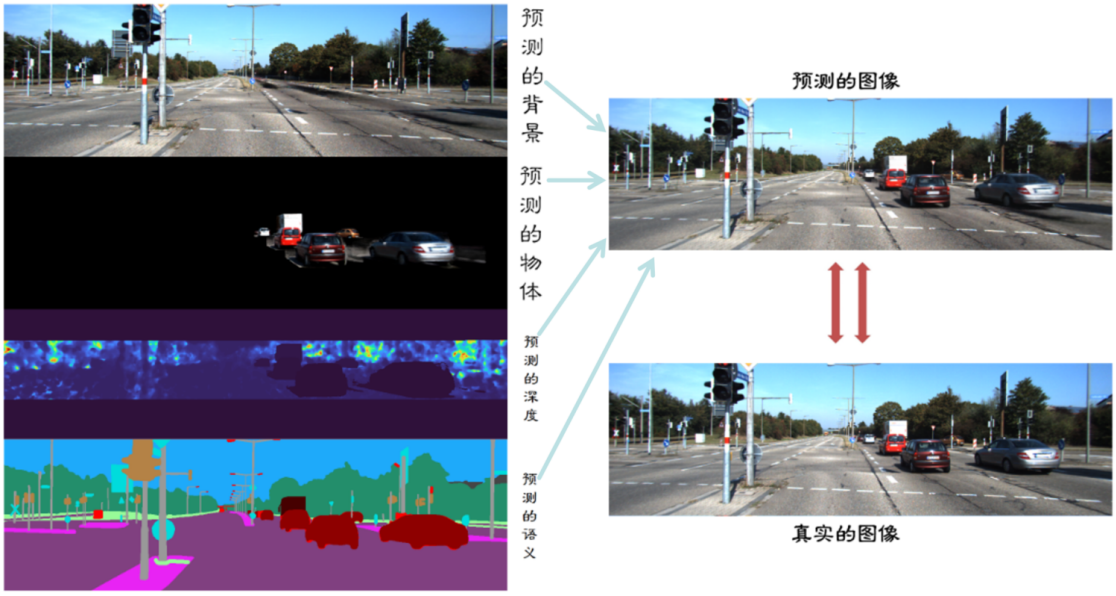


图 2: 不同的渲染组合

可以明显看出，MARS方法在各项评价指标上均优于在 KITTI 数据集上的消融测试结果 [1] 。然而，由于仅使用了一个场景进行测试，可能在整个数据集上的表现没有如此显著。此外，在观察渲染视频时，我发现汽车从近到远会出现明显的锯齿现象，这可能是由于MARS在处理远距离对象时存在一定的渲染局限性。值得注意的是，训练集上的 PSNR 达到了 31.592，比评估结果高出约 3。查阅了大量资料发现，这种现象在深度学习模型中是常见的，主要原因可能包括以下几个方面：

1. **过拟合（Overfitting）**：模型在训练过程中可能记住了训练数据的细节和噪声，从而在训练数据上表现优异，但在测试数据上表现较差。过拟合可以通过增加正则化、使用数据增强技术以及采用早停策略来缓解。

2. **数据多样性不足**：仅使用一个场景进行训练和测试，可能导致模型缺乏对不同场景的泛化能力。增加训练数据的多样性，使用更多不同场景的数据进行训练，可以提升模型的泛化性能。

3. **训练与测试数据分布差异**：训练数据和测试数据可能在分布上存在差异，从而导致模型在测试数据上的性能下降。通过仔细检查数据的分布，并确保训练和测试数据的分布尽量一致，可以减少这种差异的影响。

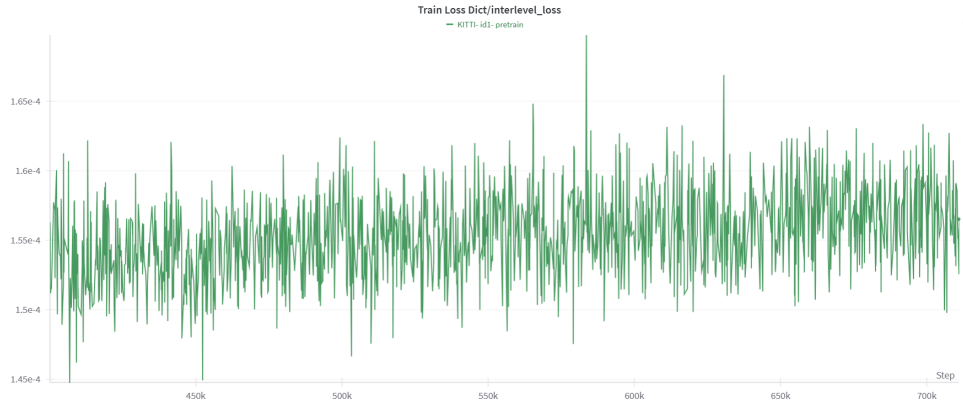


图 3: 层间损失

在分析所有损失函数的波动曲线时，我注意到层间损失的曲线呈现出缓慢波动上升的趋势，如图3所示。这与其他损失函数的波动模式不同，表明层间损失在训练过程中逐渐增加。在查阅资料后发现这种现象是正常的，只要层间损失保持在较低的值即可。层间损失是针对nerfacto方法的，跟这个有关的有 MipNeRF-360 或 ZipNeRF 的相关研究。MipNeRF-360 提出了一种多尺度表示法，通过在不同尺度上对图像进行采样来减少层间损失。而 ZipNeRF 则通过引入特征压缩技术，有效地减少了层间损失，并提高了模型的训练效率。

由于渲染结果的阴影效果十分逼真，即使在移动物体时，阴影仍然保持高度真实感。为了实现这一效果，作者并未使用独立的阴影模型。相反，阴影效果是与汽车模型一起训练的，并与背景分离。作者认为，共享类别级对象的表示有助于模型更好地分解阴影和地面外观。在我们使用实例对象表示时，阴影区域的编辑性能确实有所下降。此外，在代码实现中，通过掩码机制计算语义和天空区域的损失。

基于这些观察，可以得出结论：结合类别级对象表示进行训练，能够更有效地分离和渲染阴影和地面外观，从而提高整体的渲染效果。特别是在阴影区域，实例对象表示有所局限。

根据上述结果，我分别进行了四组消融实验，以更好地复现论文进行对比。值得注意的是，论文作者并未提供 KITTI 模型实现语义损失的具体配置。在附录 D 中，我详细列出了四组消融实验的具体模型配置及代码解释。

通过对这四组实验的比较，可以评估不同配置对模型性能的影响，以下是配置的简单概括:

- 包含语义信息的模型(`VKITTI_Recon_Mars_Car_Depth_ Semantic` 和 `KITTI_Recon_Mars_Car_Depth_ Semantic`) 可以通过语义损失来提高模型的理解能力，尤其是在复杂场景下。

- 不包含语义信息的模型 (`VKITTI_Recon_Mars_Car_Depth` 和 `KITTI_Recon_Mars_Car_Depth`) 则侧重于评估深度和车辆潜在表示对重建质量的影响。

ID	Settings						KITTI			V-KITTI		
	Model	Sampler	Category	$\mathcal{L}_{sky}$	$\mathcal{L}_{depth}$	$\mathcal{L}_{sem}$	$\mathcal{L}_{accum}$	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	LPIPS ↓
1*	Grid / Ours	prop / c2f †						**25.04**	**0.782**	**0.175**	**28.37**	**0.907**	**0.108**
2	**MLP** / Ours	**c2f** / c2f						20.14	0.589	0.476	22.19	0.664	0.409
3	Grid / Ours	prop / c2f					×	21.35	0.713	0.242	27.30	0.881	0.130
4	Grid / Ours	prop / c2f		×		×		23.68	0.774	0.181	27.32	0.881	0.129
4	Grid / Ours	prop / c2f		×		×		**28.27**	**0.896**	**0.056**	**27.43**	**0.880**	**0.114**
5	Grid / Ours	prop / c2f			×			23.66	0.769	0.184	27.30	0.880	0.128
6	Grid / Ours	prop / c2f		×				20.07	0.723	0.251	27.42	0.863	0.148
6	Grid / Ours	prop / c2f		×				**18.60**	**0.589**	**0.402**	**20.44**	**0.631**	**0.373**
7	Grid / **MLP**	prop / **c2f**						20.46	0.709	0.255	26.46	0.875	0.132
8	Grid / **Grid**	prop / **prop**						22.23	0.741	0.211	25.22	0.871	0.134
9	Grid / **MLP**	prop / **c2f**	×					20.98	0.699	0.257	27.27	0.881	0.130
10	Grid / **Grid**	prop / **prop**	×					23.71	0.763	0.193	26.65	0.882	0.125
11*	**MLP** / **MLP**	**c2f** / **c2f**						20.42	0.592	0.472	21.77	0.659	0.410

† prop 代表 proposal sampler, c2f 代表 coarse-to-fine sampler
* ID 1 是我们的默认设置
* 红色字体代表在KITTI场景06下迭代30万次
* 蓝色字体代表在V-KITTI场景02下迭代20万次

根据上表内容，针对消融实验部分，由于原始数据集过大，训练时间较长，因此缩小了训练数据，以便在单一场景下能较快得出结论。我们可以清楚地看到，添加语义损失后的效果明显更差。在 KITTI 和 V-KITTI 两个数据集上，模型在没有语义损失的情况下取得了更好的 PSNR、SSIM 和 LPIPS。例如，在 ID4 中，当添加了语义损失时， KITTI 数据集的 PSNR 达到了最高的 28.27 ，而 SSIM 和 LPIPS 分别达到了 0.896 和 0.056，显示了显著的性能提升。而在其他设置中，添加语义损失的模型表现不如不添加语义损失的模型，反映了语义损失在不同情况下对模型性能的影响，这明显与我们的直觉相违背。导致这一情况的原因可能是天空区域由于其复杂的光照和云层变化，可能会在语义分割中产生较多误差，进一步加剧这种冲突。

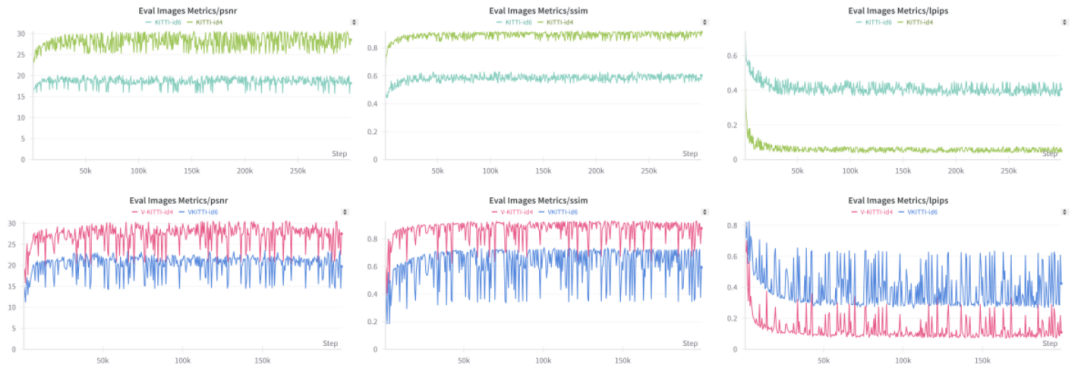


图 4: 评价指标图

如图4所示，Mars 模型在迭代次数增加的过程中逐渐趋于收敛。在 VKITTI-02 场景中，各图像间的指标波动幅度较大。这主要归因于原始 VKITTI-02 场景的图像复杂性显著高于 KITTI-06 场景。此外，KITTI 和 VKITTI 场景的消融实验数据已上传至 WandB 平台，详情请参考: MARS模型-KITTI报告 和 MARS模型-VKITTI报告。

在 Github 上，我给出了所有的渲染视频结果，详情请点击：Mars_reproduction。

1.4 感知实例编辑

感知实例编辑主要通过修改cicai_render.py文件实现，核心代码如下所示：

```
150   pose[:, :, 0, 2] = pose[:, :, 0, 2]
151   rotation[:, :, 0] = rotation[:, :, 0]
```

通过以上代码，我们可以操作场景中物体的平移和旋转矩阵。平移矩阵 pose 的后两列分别代表 object_id 和 axis，而旋转矩阵 rotation 的最后一列代表 object_id。其中，axis 从 0-2 分别代表前进、上下和左右。我们可以通过选择不同的组合来表达不同的场景变化。例如，通过添加以下代码，我们可以实现特定 id 物体的删除（图5）：

```
1   track_ids = batch_obj_dyn[..., 4]
2   track_ids[:,:,track_id] = 0
```



图 5: 物体删除

以KITTI-0006场景为例：



图 6: 感知实例编辑

如图6所示，第一行演示了全部物体不同的平移方式，在运行过程中可以发现当左右方向增加的幅度从 0.3 下降到 0.1，渲染效果明显更好，并且渲染时间同时下降。第二行演示了全部物体不同的旋转幅度，当旋转幅度越小，渲染效果越好。第三行展示了对某一 id 车辆的组合编辑。

1.5 潜在改进方向与建议

如图7所示，在使用 NuScenes 数据集中的场景 0106 进行评估时，相机姿态和物体边界框之间存在明显的不匹配。在训练过程中，通过评估图像中的粗略边界框位置可以观察到这种不匹配现象。作者本人

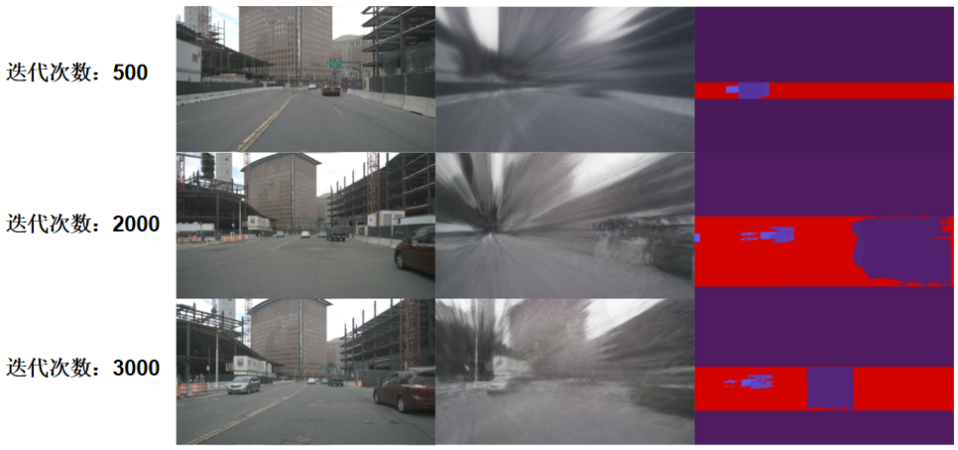


图 7: MARS在NuScenes数据集上的迭代结果#issues42

则提出:"We have tried MARS on the nuScenes dataset, and we found that the camera & object pose from the dataset are extremely noisy. One potential fix is to run a SLAM method to refine all the poses, but we haven't tried that yet."

针对该问题，一个潜在的改进方向是：运行同步定位与地图构建（SLAM）方法来优化和校正相机和物体姿态，减少噪声和不匹配现象。同时，对 NuScenes 数据进行校准和预处理，以确保数据的一致性和准确性。开发检查和验证工具，及时发现和修正姿态估计中的错误，并根据数据特点调整模型的训练参数，以更好地适应数据中的噪声和不确定性。



图 8: MARS在DAIR-V2X数据集上的评估#issues74

除了原文中提到的MARS未能考虑玻璃或其他材料上的动态镜面效应外，Mars在渲染雨天带有波纹的图像8时，明显可以看到伪影的出现。这些伪影在高分辨率图像放大后尤为明显，导致图像质量下降。作者本人则提出:"seems to be a typical type of artifact of nerfacto, no good insight on that so far to improve the rendering quality. However, you can further optimize the NeRF rendered images with post-optimizers like NeRFLiX or so." 针对该问题，我们可以引入NeRFLiX等后处理优化器，通过对渲染图像进行细化处理，减少波纹伪影，提高图像的真实感。

1.6 总结

在本次论文复现过程中，由于时间和计算资源的限制，无法使用全部数据集进行大量迭代。然而，实验仍然取得了一些有趣的发现和收获。在实验过程中我观察到了一些反直觉的结果。例如，层间损失随着迭代次数的增加而缓慢波动上升，训练集上的PSNR与评估结果之间存在3个单位的“巨大”差距，共享类别级对象表示比实例级对象表示的效果更好等等。通过这次实验，我还学会了如何使用 wanb 平台上传和可视化所有数据。这极大地提高了实验的透明度和结果的可追溯性，使我们能够更直观地分析

和理解实验数据，发现潜在的问题和改进的方向。

在整个过程中，最值得惊喜的是感知实例编辑这块内容，这让我联想到了最近的新闻，萝卜快跑在武汉的落地，无人驾驶技术已然走进现实。尽管实例编辑幅度增大时渲染效果会有所下降，但我相信在不久的将来，这项技术将在解决剩余的极端情况 (remaining corner cases) 上发挥关键作用。

报告链接：

MARS模型-KITTI报告

MARS模型-VKITTI报告

MARS模型-汇总报告

代码链接：

Mars_reproduction

References

[1] Z. Wu et al., "Mars: An instance-aware, modular and realistic simulator for autonomous driving," Artificial Intelligence, pp. 3–15, 2024. doi: 10.1007/978-981-99-8850-1_1.

[2] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, "Mip-NeRF 360: Unbounded Anti-Aliased Neural Radiance Fields," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 5460-5469, doi: 10.1109/CVPR52688.2022.00539.

[3] T. Liu, H. Zhao, Y. Yu, G. Zhou and M. Liu, "Car-Studio: Learning Car Radiance Fields From Single-View and Unlimited In-the-Wild Images," in IEEE Robotics and Automation Letters, vol. 9, no. 3, pp. 2024-2031, March 2024, doi: 10.1109/LRA.2024.3349949.

[4] https://github.com/theNded/mini-omnidata

附录

A 附录1：环境安装

```
# Create the base environment
conda create --name mars -y python=3.9
conda activate mars

# Configure tinycudann environment
pip install mars-nerfstudio
sudo apt-get install build-essential git
clear
sudo apt-get install build-essential git cmake
git clone --recursive https://github.com/nvlabs/tiny-cuda-nn
cd tiny-cuda-nn

# Build the project using CMake
cmake -DCMAKE_CUDA_COMPILER=/usr/local/cuda-12.4/bin/nvcc . -B build
cmake --build build --config RelWithDebInfo -j

# Install PyTorch extension
cd bindings/torch
python setup.py install
```

B 附录2：超参数详细配置

神经场景图模型（**Scene Graph Model**）的配置，包含了许多参数，这些参数用于控制模型的训练和推理过程。以下是代码中各个参数的详细解释：

类定义和基本结构

```
  class SceneGraphModelConfig(ModelConfig):
  """Neural Scene Graph Model Config"""

  _target: Type = field(default_factory=lambda: SceneGraphModel)
```

- **SceneGraphModelConfig**：继承自 ModelConfig，用于定义神经场景图模型的配置。
- **_target**：指定模型的目标类型，这里默认是 SceneGraphModel。

模型配置参数

```
  background_model: ModelConfig = NerfactoModelConfig()
  object_model_template: ModelConfig = NerfactoModelConfig()
```

- **background_model**：背景模型的配置，使用 NerfactoModelConfig。
- **object_model_template**：对象模型的模板配置，使用 NerfactoModelConfig。

场景相关参数

```
max_num_obj: int = -1
ray_add_input_rows: int = -1
near_plane: float = 0.05
far_plane: float = 1000.0
```

- **max_num_obj**：场景中对象的最大数量，默认值为 -1，表示没有限制。
- **ray_add_input_rows**：光线附加输入行的数量，默认值为 -1，表示没有附加输入行。
- **near_plane**：光线采样的起始位置，单位是米。
- **far_plane**：光线采样的结束位置，单位是米。

背景颜色配置

```
background_color: Literal["random", "last_sample", "black"] = "black"
```

- **background_color**：背景颜色配置，可以是 ”random”、”last_sample” 或 ”black”。

潜在向量和损失权重

```
latent_size: int = 256
orientation_loss_mult: float = 0.0001
pred_normal_loss_mult: float = 0.001
predict_normals: bool = False
```

- **latent_size**：每个类别对象的潜在向量的大小。0 表示不使用潜在向量。
- **orientation_loss_mult**：方向损失的乘数，用于计算法线的损失。
- **pred_normal_loss_mult**：预测法线损失的乘数。
- **predict_normals**：是否预测法线。

对象表示和采样策略

```
object_representation: Literal["class-wise", "object-wise"] = "object
    ↪ -wise"
object_ray_sample_strategy: Literal["warmup", "remove-bg", "none"] =
    ↪ "remove-bg"
object_warmup_steps: int = 1000
```

- **object_representation**：对象的表示方式，可以是 ”class-wise”（按类别）或 ”object-wise”（按对象）。

- **object_ray_sample_strategy**：对象射线采样策略，可以是 ”warmup”、”remove-bg” 或 ”none”。
- **object_warmup_steps**：对象模型的预热步数。

深度和语义损失配置

```
depth_loss_mult: float = 1e-2
semantic_loss_mult: float = 1.0
mono_depth_loss_mult: float = 0.0
is_euclidean_depth: bool = False
depth_sigma: float = 0.05
```

- **depth_loss_mult**：深度损失的乘数。

- **semantic_loss_mult**：语义损失的乘数。
- **mono_depth_loss_mult**：单目深度一致性损失的乘数。
- **is_euclidean_depth**：输入的深度图是否为欧几里得距离。
- **depth_sigma**：深度值的不确定性，单位是米。

深度衰减配置

```
should_decay_sigma: bool = False
starting_depth_sigma: float = 4.0
sigma_decay_rate: float = 0.99980
```

- **should_decay_sigma**：是否指数衰减 sigma。
- **starting_depth_sigma**：初始深度值的不确定性，单位是米。
- **sigma_decay_rate**：指数衰减速率。

深度损失类型

```
depth_loss_type: DepthLossType = DepthLossType.DS_NERF
use_interlevel_loss: bool = True
interlevel_loss_mult: float = 1.0
```

- **depth_loss_type**：深度损失的类型，默认是 DS-NeRF。
- **use_interlevel_loss**：是否使用 interlevel 损失。
- **interlevel_loss_mult**：interlevel 损失的乘数。

调试和天空模型配置

```
debug_object_pose: bool = False
use_sky_model: bool = False
sky_model: Optional[SkyModelConfig] = SkyModelConfig()
```

- **debug_object_pose**：是否在调试中渲染对象的边界框。
- **use_sky_model**：是否使用天空模型。
- **sky_model**：天空模型的配置。

C 附录3：MARS Pipeline 详细解释

图 9 展示了MARS Pipeline的工作流程，分为左边的流程和右边的渲染结果。具体步骤如下：

1. **选择架构 (Select architecture)**：
   - 支持多种NeRF骨架，包括MLP-based方法和Grid-based方法。
   - 不同的采样选项如Uniform Sampler、Importance Sampler、Proposal Sampler等。
2. **采样与前向传递 (Sample & Forward-pass)**：
   - **背景节点 (Scene Node)**：
     - 计算查询光线r与所有可见实例边界框的相交点，得到进入和离开距离（[tin, tout]）。
     - 使用选择的场景表示模型和采样方法来推断点属性，如RGB、密度、语义等。
   - **前景节点 (Object Node)**：
     - 光线首先被转换到实例坐标系，然后通过前景节点的表示模型处理。
3. **组合渲染 (Compositional Rendering)**：
   - 所有从背景和前景节点采样的点被组合并进行体积渲染，以生成RGB图像、深度图和语义分割图。

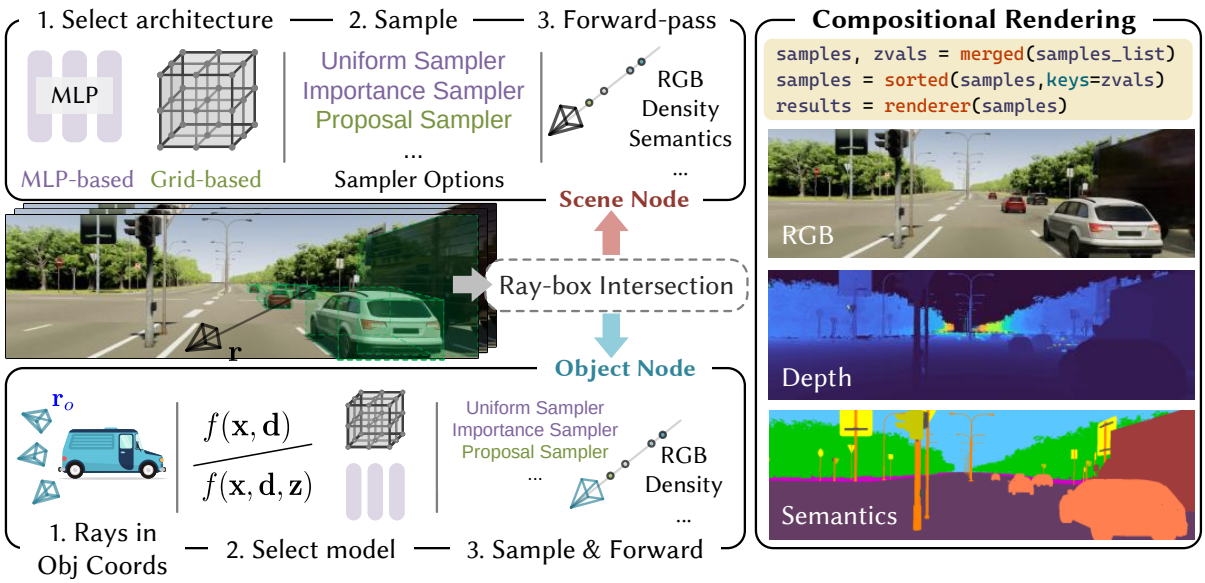


图 9: MARS Pipeline

代码解析

MarsPipelineConfig 类

```
@dataclass
class MarsPipelineConfig(VanillaPipelineConfig):
"""Configuration for pipeline instantiation"""
_target: Type = field(default_factory=lambda: MarsPipeline)
datamanager: DataManagerConfig = VanillaDataManagerConfig()
model: ModelConfig = ModelConfig()
```

- `MarsPipelineConfig` 定义了Pipeline的配置，包括数据管理器和模型配置。

MarsPipeline 类

```
class MarsPipeline(Pipeline):
def __init__(
self,
config: MarsPipelineConfig,
device: str,
test_mode: Literal["test", "val", "inference"] = "val",
world_size: int = 1,
local_rank: int = 0,
grad_scaler: Optional[GradScaler] = None,
):
super().__init__()
self.config = config
self.test_mode = test_mode
self.datamanager: MarsDataManager = config.datamanager.setup(
```

```
    device=device, test_mode=test_mode, world_size=world_size, local_rank
        ↪ =local_rank
    )
    self.datamanager.to(device)
    assert self.datamanager.train_dataset is not None, "Missing input
        ↪ dataset"
    self.config.model.max_num_obj = self.config.datamanager.dataparser.
        ↪ max_input_objects
    self.config.model.ray_add_input_rows = self.config.datamanager.
        ↪ dataparser.add_input_rows
    self.train_obj_info = self.datamanager.train_dataset.metadata["obj_
        ↪ info"]
    self.eval_obj_info = self.datamanager.eval_dataset.metadata["obj_info
        ↪ "]

    use_car_latents = "car_latents" in self.datamanager.train_dataset.
        ↪ metadata
    self._model = config.model.setup(
    scale_factor=self.datamanager.train_dataset.metadata["scale_factor"],
    scene_box=self.datamanager.train_dataset.scene_box,
    num_train_data=len(self.datamanager.train_dataset),
    metadata=self.datamanager.train_dataset.metadata,
    device=device,
    grad_scaler=grad_scaler,
    object_meta=self.datamanager.train_dataset.metadata,
    obj_feat_dim=None,
    use_car_latents=use_car_latents,
    car_latents=self.datamanager.train_dataset.metadata["car_latents"] if
        ↪  use_car_latents else None,
    car_nerf_state_dict_path=self.datamanager.train_dataset.metadata["car
        ↪ _nerf_state_dict_path"]
    if use_car_latents else None,
    use_depth=self.datamanager.train_dataset.use_depth,
    use_semantic=self.datamanager.train_dataset.use_semantic,
    semantics_meta=self.datamanager.train_dataset.metadata["semantics"]
    if self.datamanager.train_dataset.use_semantic else None,
    )
    self.model.to(device)

    self.world_size = world_size
    if world_size > 1:
    self._model = typing.cast(Model, DDP(self._model, device_ids=[local_
        ↪ rank], find_unused_parameters=True))
```

```
48  dist.barrier(device_ids=[local_rank])
```

初始化

1. **初始化配置**：将配置文件传递给MarsPipeline实例，并根据配置设置数据管理器MarsDataManager。

2. **设置数据管理器**：根据配置文件设置数据管理器，包括训练和评估数据集的元数据。
3. **初始化模型**：根据配置文件初始化模型，并设置模型的各种参数，如缩放因子、场景框、训练数据数量、元数据等。
4. **多设备支持**：如果存在多个设备，使用分布式数据并行（DDP）包装模型，并同步设备。

获取训练损失字典

```
1   @profiler.time_function
2   def get_train_loss_dict(self, step: int):
3   ray_bundle, batch = self.datamanager.next_train(step)
4   model_outputs = self.model(ray_bundle)
5   metrics_dict = self.model.get_metrics_dict(model_outputs, batch)
6   if self.config.datamanager.camera_optimizer is not None:
7   camera_opt_param_group = self.config.datamanager.camera_optimizer.
        ↪ param_group
8   if camera_opt_param_group in self.datamanager.get_param_groups():
9   metrics_dict["camera_opt_translation"] = (
10  self.datamanager.get_param_groups()[camera_opt_param_group][0].data
        ↪ [:, :3].norm()
11  )
12  metrics_dict["camera_opt_rotation"] = (
13  self.datamanager.get_param_groups()[camera_opt_param_group][0].data
        ↪ [:, 3:].norm()
14  )
15  loss_dict = self.model.get_loss_dict(model_outputs, batch, metrics_
        ↪ dict)
16  return model_outputs, loss_dict, metrics_dict
```

1. **获取训练数据**：从数据管理器获取下一批训练数据（光线包和批量数据）。
2. **模型前向传递**：将光线包传递给模型，获得模型输出。
3. **计算指标字典**：根据模型输出计算指标字典。
4. **记录相机优化指标**：如果配置了相机优化器，记录相机优化的指标，如相机平移和旋转的范数。
5. **计算损失字典**：根据模型输出和指标字典计算损失字典。

获取评估损失字典

```
1   @profiler.time_function
```

```
    def get_eval_loss_dict(self, step: int) -> Tuple[Any, Dict[str, Any],
        ↪ Dict[str, Any]]:
    self.eval()
    ray_bundle, batch = self.datamanager.next_eval(step)
    model_outputs = self.model(ray_bundle)
    metrics_dict = self.model.get_metrics_dict(model_outputs, batch)
    loss_dict = self.model.get_loss_dict(model_outputs, batch, metrics_
        ↪ dict)
    self.train()
    return model_outputs, loss_dict, metrics_dict
```

1. **设置评估模式**：将模型设置为评估模式。
2. **获取评估数据**：从数据管理器获取下一批评估数据（光线包和批量数据）。
3. **模型前向传递**：将光线包传递给模型，获得模型输出。
4. **计算指标和损失**：根据模型输出计算指标字典和损失字典。
5. **恢复训练模式**：将模型恢复为训练模式。

获取评估图像指标和图像

```
    @profiler.time_function
    def get_eval_image_metrics_and_images(self, step: int):
    self.eval()
    image_idx, camera_ray_bundle, batch = self.datamanager.next_eval_
        ↪ image(step)
    outputs = self.model.get_outputs_for_camera_ray_bundle(camera_ray_
        ↪ bundle)
    metrics_dict, images_dict = self.model.get_image_metrics_and_images(
        ↪ outputs, batch)
    assert "image_idx" not in metrics_dict
    metrics_dict["image_idx"] = image_idx
    assert "num_rays" not in metrics_dict
    metrics_dict["num_rays"] = len(camera_ray_bundle)
    self.train()
    return metrics_dict, images_dict
```

1. **设置评估模式**：将模型设置为评估模式。
2. **获取评估图像数据**：从数据管理器获取下一张评估图像的数据（图像索引、相机光线包和批量数据）。
3. **模型前向传递**：将相机光线包传递给模型，获得输出。
4. **计算图像指标和图像**：根据模型输出计算图像指标和图像字典。
5. **恢复训练模式**：将模型恢复为训练模式。

获取平均评估图像指标

```
@profiler.time_function
def get_average_eval_image_metrics(self, step: Optional[int] = None):
self.eval()
metrics_dict_list = []
assert isinstance(self.datamanager, VanillaDataManager)
num_images = len(self.datamanager.fixed_indices_eval_dataloader)
with Progress(
TextColumn("[progress.description]{task.description}"),
BarColumn(),
TimeElapsedColumn(),
MofNCompleteColumn(),
transient=True,
) as progress:
task = progress.add_task("[green]Evaluating all eval images...",
    ↪ total=num_images)
for camera_ray_bundle, batch in self.datamanager.fixed_indices_eval_
    ↪ dataloader:
inner_start = time()
object_rays_info = self.datamanager.eval_dataset.metadata["obj_info
    ↪ "][batch["image_idx"]]
object_rays_info_view = object_rays_info.expand(camera_ray_bundle.
    ↪ shape[0], camera_ray_bundle.shape[1], -1, -1)
camera_ray_bundle.metadata["object_rays_info"] = object_rays_info_
    ↪ view.reshape(
camera_ray_bundle.shape[0], camera_ray_bundle.shape[1], -1
).detach()
height, width = camera_ray_bundle.shape
num_rays = height * width
outputs = self.model.get_outputs_for_camera_ray_bundle(camera_ray_
    ↪ bundle)
metrics_dict, _ = self.model.get_image_metrics_and_images(outputs,
    ↪ batch)
metrics_dict["num_rays_per_sec"] = num_rays / (time() - inner_start)
fps_str = "fps"
assert fps_str not in metrics_dict
metrics_dict[fps_str] = metrics_dict["num_rays_per_sec"] / (height *
    ↪ width)
metrics_dict_list.append(metrics_dict)
progress.advance(task)
metrics_dict = {}
for key in metrics_dict_list[0].keys():
```

```
    metrics_dict[key] = float(torch.mean(torch.tensor([metrics_dict[key]
        ↪ for metrics_dict in metrics_dict_list])))
    self.train()
    return metrics_dict
```

1. **设置评估模式**：将模型设置为评估模式。
2. **初始化指标字典列表**：用于记录每张图像的评估指标。
3. **遍历评估图像数据集**：获取每张评估图像的数据（相机光线包和批量数据）。
4. **计算光线信息**：获取并扩展对象光线信息。
5. **模型前向传递**：将相机光线包传递给模型，获得输出。
6. **计算图像指标和图像**：根据模型输出计算图像指标和图像字典，并记录每秒处理的光线数。
7. **计算平均指标**：对所有图像指标计算平均值。
8. **恢复训练模式**：将模型恢复为训练模式。

加载Pipeline

```
def load_pipeline(self, loaded_state: Dict[str, Any], step: int) ->
    ↪ None:
state = {
  (key[len("module.") :] if key.startswith("module.") else key):
     ↪ value for key, value in loaded_state.items()
}
self.model.update_to_step(step)
self.load_state_dict(state, strict=True)
```

1. **加载检查点**：从给定路径加载检查点。
2. **更新模型状态**：根据加载的状态字典更新模型参数。

获取训练回调

```
def get_training_callbacks(
self, training_callback_attributes: TrainingCallbackAttributes
) -> List[TrainingCallback]:
datamanager_callbacks = self.datamanager.get_training_callbacks(
    ↪ training_callback_attributes)
model_callbacks = self.model.get_training_callbacks(training_callback
    ↪ _attributes)
callbacks = datamanager_callbacks + model_callbacks
return callbacks
```

1. **获取数据管理器的训练回调**：从数据管理器获取训练回调。
2. **获取模型的训练回调**：从模型获取训练回调。
3. **合并回调**：将数据管理器和模型的训练回调合并并返回。

获取参数组

```
def get_param_groups(self) -> Dict[str, List[Parameter]]:
datamanager_params = self.datamanager.get_param_groups()
model_params = self.model.get_param_groups()
return {**datamanager_params, **model_params}
```

1. **获取数据管理器参数组**：从数据管理器获取参数组。
2. **获取模型参数组**：从模型获取参数组。
3. **合并参数组**：将数据管理器和模型的参数组合并并返回。

D 附录4：消融实验模型配置

四组消融实验的模型配置参数如下所示:

D.1 VKITTI_Recon_Mars_Car_Depth_Semantic(ID6)

这段代码是配置一个针对 VKITTI 数据集的重建方法，结合了车辆、深度和语义信息。具体配置如下：

D.1.1 方法名称和训练配置

- **方法名称**：`mars-vkitti-car-depth-recon-semantic`
  - 该名称表明该方法使用了Mars pipeline，处理 VKITTI 数据集中的车辆、深度和语义信息。
- **训练配置**：
  - `steps_per_eval_image`：每多少步评估一次图像，这个参数控制了评估频率。
  - `steps_per_eval_all_images`：每多少步评估所有图像，这个参数决定了完整评估的频率。
  - `steps_per_save`：每多少步保存一次模型，控制模型检查点的保存频率。
  - `max_num_iterations`：最大迭代次数，控制训练的总步数。
  - `save_only_latest_checkpoint`：是否只保存最新的检查点，可以设置为True表示只保存最新的检查点，节省存储空间。
  - `mixed_precision`：是否使用混合精度训练，设置为False表示不使用混合精度。
  - `use_grad_scaler`：是否使用梯度缩放器，设置为True表示使用梯度缩放器，帮助在混合精度训练时稳定训练过程。
  - `log_gradients`：是否记录梯度信息，设置为True表示记录梯度信息，方便后续分析。

D.1.2 数据管理器配置

- **数据解析器**：`MarsVKittiDataParserConfig`
  - `use_car_latents`：是否使用车辆潜在表示，设置为True表示使用。
  - `use_depth`：是否使用深度信息，设置为True表示使用。
  - `use_semantic`：是否使用语义信息，设置为True表示使用。
  - `semantic_mask_classes`：指定哪些类作为语义掩码，这里设置为[”Van”, ”Undefined”]。
  - `car_object_latents_path`：车辆对象潜在表示的路径。
  - `split_setting`：数据集的划分设置，这里为“reconstruction”。
  - `car_nerf_state_dict_path`：车辆NeRF模型的状态字典路径。

- semantic_path：语义分割图像的路径。

D.1.3 模型配置

- 背景模型：SemanticNerfWModelConfig
  - num_proposal_iterations：提议迭代次数。
  - num_proposal_samples_per_ray：每条射线的样本数量。
  - num_nerf_samples_per_ray：每条射线的NeRF样本数量。
  - use_single_jitter：是否使用单一抖动，设置为False表示不使用。
  - semantic_loss_weight：语义损失权重，这里设置为0.1，表示语义信息在总损失中的比重。
- 物体模型模板：CarNeRFModelConfig
  - object_representation：对象表示方式，这里为类级别表示。
  - object_ray_sample_strategy：对象射线采样策略，这里为去除背景采样。

D.1.4 优化器配置

- 背景模型优化器：
  - optimizer：RAdamOptimizerConfig
    - lr：学习率，设置为1e-3。
    - eps：优化器的epsilon值，设置为1e-15。
  - scheduler：ExponentialDecaySchedulerConfig
    - lr_final：最终学习率，设置为1e-5。
    - max_steps：最大步数，可以设置为200000。
- 物体模型优化器：
  - optimizer：RAdamOptimizerConfig
    - lr：学习率，设置为5e-3。
    - eps：优化器的epsilon值，设置为1e-15。
  - scheduler：ExponentialDecaySchedulerConfig
    - lr_final：最终学习率，设置为1e-5。
    - max_steps：最大步数，可以设置为200000。

```
VKITTI_Recon_Mars_Car_Depth_Semantic = MethodSpecification(
  config=TrainerConfig(
  method_name="mars-vkitti-car-depth-recon-semantic",
  steps_per_eval_image=STEPS_PER_EVAL_IMAGE,
  steps_per_eval_all_images=STEPS_PER_EVAL_ALL_IMAGES,
  steps_per_save=STEPS_PER_SAVE,
  max_num_iterations=MAX_NUM_ITERATIONS,
  save_only_latest_checkpoint=True,
  mixed_precision=False,
  use_grad_scaler=True,
  log_gradients=True,
  pipeline=MarsPipelineConfig(
  datamanager=MarsDataManagerConfig(
  dataparser=MarsVKittiDataParserConfig(
```

```
    use_car_latents=True,
    use_depth=True,
    use_semantic=True,
    semantic_mask_classes=["Van", "Undefined"],
    car_object_latents_path=Path(
    "latent_codes/vkitti/car-object-latents/latent_codes02.pt"
    ),
    split_setting="reconstruction",
    car_nerf_state_dict_path=Path("latent_codes/vkitti/car-nerf-state-dict/epoch_805.ckpt"),
    semantic_path=Path("datasets/vkitti/Scene02/clone/frames/classSegmentation"),
    ),
    train_num_rays_per_batch=4096,
    eval_num_rays_per_batch=4096,
    camera_optimizer=CameraOptimizerConfig(mode="off"),
    ),
    model=SceneGraphModelConfig(
    background_model=SemanticNerfWModelConfig(
    num_proposal_iterations=1,
    num_proposal_samples_per_ray=[48],
    num_nerf_samples_per_ray=97,
    use_single_jitter=False,
    semantic_loss_weight=0.1,
    ),
    object_model_template=CarNeRFModelConfig(_target=CarNeRF),
    object_representation="class-wise",
    object_ray_sample_strategy="remove-bg",
    ),
    ),
    optimizers={
      "background_model": {
        "optimizer": RAdamOptimizerConfig(lr=1e-3, eps=1e-15),
        "scheduler": ExponentialDecaySchedulerConfig(lr_final=1e-5, max_steps=100000),
      },
      "object_model": {
        "optimizer": RAdamOptimizerConfig(lr=5e-3, eps=1e-15),
        "scheduler": ExponentialDecaySchedulerConfig(lr_final=1e-5, max_steps=100000),
      },
    },
    # viewer=ViewerConfig(num_rays_per_chunk=1 << 15),
```

```
  vis="wandb",
  ),
  description="Neural Scene Graph with semantic learning for the
      ↪ backgruond model."
)
```

D.2 VKITTI_Recon_Mars_Car_Depth(ID4)

这个方法配置了一个基于NeRF的重建方法，专注于虚拟KITTI数据集中的车辆和深度信息，不包含语义信息。

D.2.1 方法名称和训练配置

- **方法名称**：`mars-vkitti-car-depth-recon`
  - 表示该方法处理VKITTI数据集中的车辆和深度信息。
- **训练配置**：
  - 与前一个方法类似，`save_only_latest_checkpoint`设置为False，表示不只保存最新的检查点，保留所有检查点。
  - 其他配置如每步评估图像数量、最大迭代次数等相同。

D.2.2 数据管理器配置

- **数据解析器**：`MarsVKittiDataParserConfig`
  - 与前一个方法类似，但不包含语义信息相关配置。

D.2.3 模型配置

- **背景模型**：`NerfactoModelConfig`
  - 这是一个基本的NeRF模型配置，不涉及语义信息。
- **物体模型模板**：`CarNeRFModelConfig`
  - 与前一个方法类似。

D.2.4 优化器配置

- **背景模型优化器**：`RAdamOptimizerConfig`，与前一个方法相同。
- **物体模型优化器**：`RAdamOptimizerConfig`，与前一个方法相同。

```
VKITTI_Recon_Mars_Car_Depth = MethodSpecification(
  config=TrainerConfig(
  method_name="mars-vkitti-car-depth-recon",
  steps_per_eval_image=STEPS_PER_EVAL_IMAGE,
  steps_per_eval_all_images=STEPS_PER_EVAL_ALL_IMAGES,
  steps_per_save=STEPS_PER_SAVE,
  save_only_latest_checkpoint=False,
  max_num_iterations=MAX_NUM_ITERATIONS,
  mixed_precision=False,
```

```
    use_grad_scaler=True,
    log_gradients=True,
    pipeline=MarsPipelineConfig(
    datamanager=MarsDataManagerConfig(
    dataparser=MarsVKittiDataParserConfig(
    use_car_latents=True,
    use_depth=True,
    car_object_latents_path=Path(
    "latent_codes/vkitti/car-object-latents/latent_codes02.pt"
    ),
    split_setting="reconstruction",
    car_nerf_state_dict_path=Path("latent_codes/vkitti/car-nerf-state-
        ↪ dict/epoch_805.ckpt"),
    ),
    train_num_rays_per_batch=4096,
    eval_num_rays_per_batch=4096,
    camera_optimizer=CameraOptimizerConfig(mode="off"),
    ),
    model=SceneGraphModelConfig(
    background_model=NerfactoModelConfig(),
    object_model_template=CarNeRFModelConfig(_target=CarNeRF),
    object_representation="class-wise",
    object_ray_sample_strategy="remove-bg",
    ),
    ),
    optimizers={
      "background_model": {
        "optimizer": RAdamOptimizerConfig(lr=1e-3, eps=1e-15),
        "scheduler": ExponentialDecaySchedulerConfig(lr_final=1e-5,
            ↪ max_steps=100000),
      },
      "object_model": {
        "optimizer": RAdamOptimizerConfig(lr=5e-3, eps=1e-15),
        "scheduler": ExponentialDecaySchedulerConfig(lr_final=1e-5,
            ↪ max_steps=100000),
      },
    },
    vis="wandb",
    ),
    description="Neural Scene Graph implementation with vanilla-NeRF
        ↪ model for backgruond and object models.",
)
```

D.3 KITTI_Recon_Mars_Car_Depth_Semantic(ID6)

这个方法配置了一个基于NeRF的重建方法，专门用于KITTI数据集中的车辆、深度和语义信息。

D.3.1 方法名称和训练配置

- 方法名称：`mars-kitti-car-depth-recon-semantic`
  - 表示该方法结合了Mars pipeline，处理KITTI数据集中的车辆、深度和语义信息。
- 训练配置：
  - 与第一个方法类似，但针对KITTI数据集进行了特定调整。

D.3.2 数据管理器配置

- 数据解析器：`MarsKittiDataParserConfig`
  - 使用了与VKITTI类似的配置，但路径和一些特定设置根据KITTI数据集进行了调整。

D.3.3 模型配置

- 背景模型：`SemanticNerfWModelConfig`
  - 与第一个方法类似。
- 物体模型模板：`CarNeRFModelConfig`
  - 与前一个方法类似。

D.3.4 优化器配置

- 背景模型优化器：`RAdamOptimizerConfig`，与前一个方法相同。
- 物体模型优化器：`RAdamOptimizerConfig`，与前一个方法相同。

```
KITTI_Recon_Mars_Car_Depth_Semantic = MethodSpecification(
  config=TrainerConfig(
  method_name="mars-kitti-car-depth-recon_semantic",
  steps_per_eval_image=STEPS_PER_EVAL_IMAGE,
  steps_per_eval_all_images=STEPS_PER_EVAL_ALL_IMAGES,
  steps_per_save=STEPS_PER_SAVE,
  max_num_iterations=MAX_NUM_ITERATIONS,
  save_only_latest_checkpoint=False,
  mixed_precision=False,
  use_grad_scaler=True,
  log_gradients=True,
  pipeline=MarsPipelineConfig(
  datamanager=MarsDataManagerConfig(
  dataparser=MarsKittiDataParserConfig(
  use_car_latents=True,
  use_depth=True,
  use_semantic=True,
  semantic_mask_classes=["van","Undefined"],
  car_object_latents_path=Path(
```

```
    "latent_codes/KITTI-MOT/car-object-latents/latent_codes06.pt"
    ),
    split_setting="reconstruction",
    car_nerf_state_dict_path=Path("latent_codes/KITTI-MOT/car-nerf-state-
        ↪ dict/epoch_670.ckpt"),
    semantic_path=Path("datasets/KITTI_MOT/panoptic_maps"),
        ↪ scale_factor=0.1
    ),
    train_num_rays_per_batch=4096,
    eval_num_rays_per_batch=4096,
    camera_optimizer=CameraOptimizerConfig(mode="off"),
    ),
    model=SceneGraphModelConfig(
    background_model=SemanticNerfWModelConfig(
    num_proposal_iterations=1,
    num_proposal_samples_per_ray=[48],
    num_nerf_samples_per_ray=97,
    use_single_jitter=False,
    semantic_loss_weight=0.1,
    ),
    object_model_template=CarNeRFModelConfig(_target=CarNeRF),
    object_representation="class-wise",
    object_ray_sample_strategy="remove-bg",
    ),
    ),
    optimizers={
      "background_model": {
        "optimizer": RAdamOptimizerConfig(lr=1e-3, eps=1e-15),
        "scheduler": ExponentialDecaySchedulerConfig(lr_final=1e-5,
            ↪ max_steps=100000),
      },
      "learnable_global": {
        "optimizer": RAdamOptimizerConfig(lr=1e-3, eps=1e-15),
        "scheduler": ExponentialDecaySchedulerConfig(lr_final=1e-5,
            ↪ max_steps=100000),
      },
      "object_model": {
        "optimizer": RAdamOptimizerConfig(lr=5e-3, eps=1e-15),
        "scheduler": ExponentialDecaySchedulerConfig(lr_final=1e-5,
            ↪ max_steps=100000),
      },
    },
    vis="wandb",
```

```
  ),
  description="Neural Scene Graph implementation with vanilla-NeRF
     ↪ model for backgruond and object models.",
)
```

D.4 KITTI_Recon_Mars_Car_Depth(ID4)

这个方法配置了一个基于NeRF的重建方法，用于KITTI数据集中的车辆和深度信息，不包含语义信息。

D.4.1 方法名称和训练配置

- 方法名称：`mars-kitti-car-depth-recon`
  - 表示该方法主要处理KITTI数据集中的车辆和深度信息。
- 训练配置：
  - 与前一个方法类似，但 `save_only_latest_checkpoint` 设置为False，表示不只保存最新的检查点，保留所有检查点。

D.4.2 数据管理器配置

- 数据解析器：`MarsKittiDataParserConfig`
  - 与前一个方法类似，但不包括语义信息的处理。

D.4.3 模型配置

- 背景模型：`NerfactoModelConfig`
  - 不涉及语义信息。
- 物体模型模板：`CarNeRFModelConfig`
  - 与前一个方法类似。

D.4.4 优化器配置

- 背景模型优化器：`RAdamOptimizerConfig`，与前一个方法相同。
- 物体模型优化器：`RAdamOptimizerConfig`，与前一个方法相同。

```
KITTI_Recon_Mars_Car_Depth = MethodSpecification(
  config=TrainerConfig(
  method_name="mars-kitti-car-depth-recon",
  steps_per_eval_image=STEPS_PER_EVAL_IMAGE,
  steps_per_eval_all_images=STEPS_PER_EVAL_ALL_IMAGES,
  steps_per_save=STEPS_PER_SAVE,
  max_num_iterations=MAX_NUM_ITERATIONS,
  save_only_latest_checkpoint=False,
  mixed_precision=False,
  use_grad_scaler=True,
  log_gradients=True,
```

```
  pipeline=MarsPipelineConfig(
  datamanager=MarsDataManagerConfig(
  dataparser=MarsKittiDataParserConfig(
  use_car_latents=True,
  use_depth=True,
  car_object_latents_path=Path(
  "latent_codes/KITTI-MOT/car-object-latents/latent_codes06.pt"
  ),
  split_setting="reconstruction",
  car_nerf_state_dict_path=Path("latent_codes/KITTI-MOT/car-nerf-state-
     ↪ dict/epoch_670.ckpt"),
  scale_factor=0.1,
  ),
  train_num_rays_per_batch=4096,
  eval_num_rays_per_batch=4096,
  camera_optimizer=CameraOptimizerConfig(mode="off"),
  ),
  model=SceneGraphModelConfig(
  background_model=NerfactoModelConfig(),
  object_model_template=CarNeRFModelConfig(_target=CarNeRF),
  object_representation="class-wise",
  object_ray_sample_strategy="remove-bg",
  mono_depth_loss_mult=0.01,
  depth_loss_mult=0,
  ),
  ),
  optimizers={
    "background_model": {
      "optimizer": RAdamOptimizerConfig(lr=1e-3, eps=1e-15),
      "scheduler": ExponentialDecaySchedulerConfig(lr_final=1e-5,
         ↪ max_steps=200000),
    },
    "learnable_global": {
      "optimizer": RAdamOptimizerConfig(lr=1e-3, eps=1e-15),
      "scheduler": ExponentialDecaySchedulerConfig(lr_final=1e-5,
         ↪ max_steps=200000),
    },
    "object_model": {
      "optimizer": RAdamOptimizerConfig(lr=5e-3, eps=1e-15),
      "scheduler": ExponentialDecaySchedulerConfig(lr_final=1e-5,
         ↪ max_steps=200000),
    },
  },
```

```
    vis="wandb",
    ),
    description="Neural Scene Graph implementation with vanilla-NeRF
        ↪ model for backgruond and object models.",
)
```